

RAG fact checker project report

Михаил Преображенский

8 апреля 2026 г.

Содержание

1	Introduction	2
2	Идея проекта	2
3	Данные	2
4	Обучение	2
4.1	RAG	2
4.1.1	Эмбеддинги и FAISS	2
4.1.2	Разбиение Википедии на чанки	3
4.1.3	Валидация ретрива	3
4.2	Verifier model	3
4.2.1	Логика работы модели	3
4.2.2	Обучение	4
5	Итог	5

1 Introduction

После прочтения Антоном лекции про HF и другие доступные нам инструменты, у меня появилось сильное желание сделать что-то большее, чем просто "классификатор статей". Для меня HF выглядел как огромный шведский стол всяких моделей, из которых можно собрать что-то свое, рабочее — это и стало отправной точкой моего "провального" проекта, потому что все (внезапно) оказалось не так просто.

2 Идея проекта

Мной предполагалось построение полноценной RAG-системы по Википедии, которая бы в ответ на поступившее утверждение (**claim**) возвращала наиболее релевантную информацию (**evidence**). Это все поступало бы в BERT-классификатор, и уже он, на основе извлеченной информации и утверждения, давал бы ему искомый показатель (**truthfulness**):

$$truthfulness = \frac{p_{supports}}{p_{supports} + p_{refutes}}.$$

На словах все звучало примитивно. На деле - not so good.

3 Данные

За данными я, как полагается, тоже отправился на HF. Было принято решение использовать только [английский дамп википедии](#) (23гб), так как она имеет наибольшее покрытие информации, а multi-language модель обучать значительно сложнее (еще одна потенциальная point of failure).

В качестве датасета на обучение claim-evidence поведения я выбрал датасет [fever](#) специально обученный на дампе википедии 17 года. Состоит из утверждений, либо подтверждаемых информацией из вики (тогда есть ссылка "Golden title" на статью источник), либо опровергаемых, либо не имеющих достаточно информации в конкретном дампе. Одной из моих ошибок, в итоге, стало, что я подумал, что можно обучиться и на более новом дампе, в итоге это привело к тому что где-то половина Golden Titles не матчилась 1 в 1, и я их в итоге не использовал.

4 Обучение

4.1 RAG

4.1.1 Эмбединги и FAISS

Начал я свои эксперименты с RAG. Тщательно изучив имеющиеся в наличии эмбединги для ретрива данных на [MTEB leaderboard](#), я остановил свой выбор на недавно вышедших маловесных эмбедингах от Microsoft: [harrier-oss-v1-0.6b](#).

Но, изучив параметры планов HF spaces, я пришел к выводу, что FAISS-индекс даже такой легкой моделькой будет весить слишком много, чтобы поместиться в ОЗУ сри-планов (около 40гб при доступных 32 в CPU-UPGRADE плане). Так что я дополнительно подробно изучил возможность "квантизации" эмбедингов, встроенной в FAISS библиотеку, что уменьшило потребляемую память до уместных 7 гб.

После этого я 40 часов прогонял эмбединги на доступной мне на тот момент **gpu a100** и получил около 100гб эмбедингов, которые я [выложил в общий доступ](#), вдруг кому пригодится.

4.1.2 Разбиение Википедии на чанки

Предобработка википедии у меня была довольно примитивной: я просто разбил ее на чанки по 200 **слов** с перекрытием в 50 и прицепил к ним в начало название статьи в качестве дополнительного сигнала.

Проблемой стало хранение этого. Очевидно, в оперативную память дополнительные 23 гигабайта тоже не особо вписывались, так что я решил закинуть все данные в бдшку на основе SQLite и хранить их на жестком диске. На мое удивление, это никак не замедлило мое последующее обучение, и ретрив данных с диска показал себя очень хорошо (особенно с предзагрузкой батчей).

4.1.3 Валидация ретрива

Вот результаты одного из моих экспериментов по ретриву Golden Title на нескольких разных квантизациях 2х версий эмбединга.

Таблица 1: Retrieval Benchmark — microsoft/harrier-oss-v1-270m (dim=640, nprobe=32, claims=45033)

Индекс	Векторов	Размер	Recall@1	Recall@3	Recall@5
PQ160 (IVF4096,PQ160)	23 758 035	4,00 ГБ	32,5%	44,3%	47,4%
PQ320 (IVF4096,PQ320)	23 758 035	7,80 ГБ	34,2%	45,6%	48,2%

Таблица 2: Retrieval Benchmark — microsoft/harrier-oss-v1-0.6b (dim=1024, nprobe=32, claims=45033)

Индекс	Векторов	Размер	Recall@1	Recall@3	Recall@5
PQ128 (IVF4096,PQ128)	23 758 035	3,25 ГБ	32,5%	44,6%	47,7%
PQ256 (IVF4096,PQ256)	23 758 035	6,29 ГБ	35,5%	47,3%	50,0%

4.2 Verifier model

4.2.1 Логика работы модели

Изначально я, если честно, планировал просто склеить claim и все evidences, но я быстро столкнулся с проблемой: у большинства бертов контекст был 512 токенов, а один evidence чанк уже занимал где-то 400.

В итоге я пришел к такой схеме: составляем 5 прогонов (я остановился на ретриве топ5 релевантных чанков из вики):

$$\text{torch.tensor}[\langle \text{bos} \rangle + \text{claim} + \langle \text{sep} \rangle + \text{evidence}_i] \rightarrow [\text{logit}_{\text{sup}}, \text{logit}_{\text{ref}}, \text{logit}_{\text{nei}}]$$

и делаем макспулинг по всем 5ти логит-векторам после софтмакса.

Логика, почему maxpooling, а не meanpooling: достаточно только одного железобетонного подтверждения supports или refutes, чтобы сделать окончательный выбор.

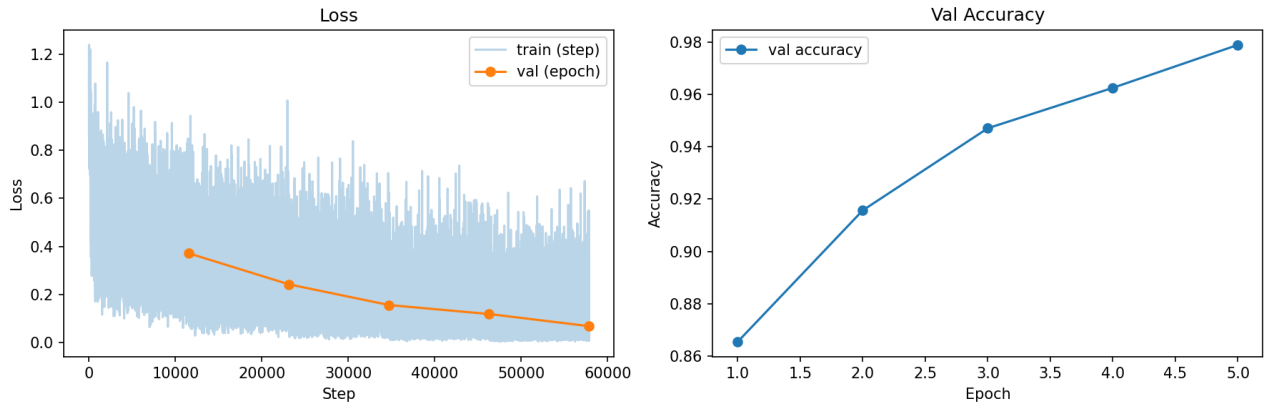
В итоге берем еще один софтмакс от полученного вектора и получаем окончательные вероятности 3х исходов.

Очевидно, что у такой модели есть недостатки: к примеру, вообще никак нельзя будет детерминированно учитывать зависимости между evidences: «вася — кот» и «все коты -

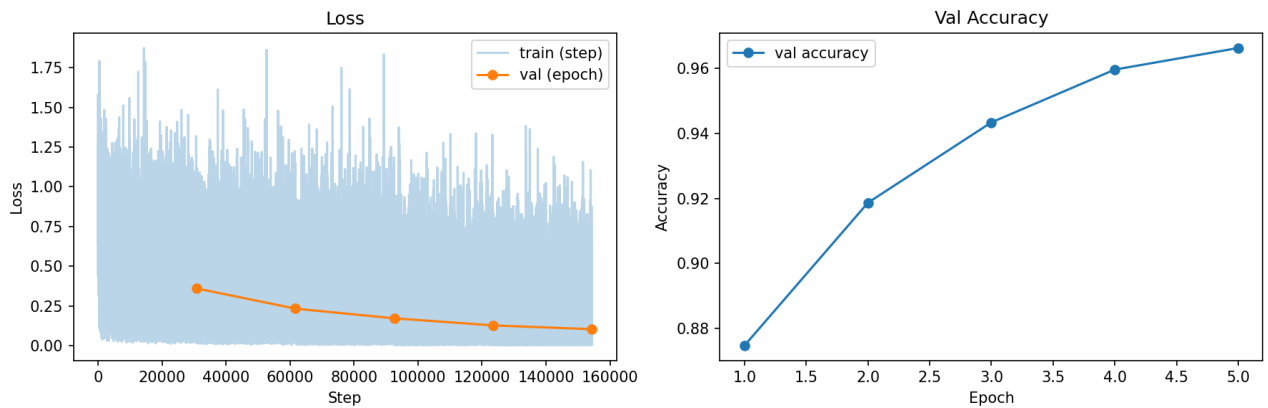
рыжие» вообще не обязательно приведут к «Вася — рыжий», так как будут проходить через берг по отдельности. Но, к сожалению, это жертва, на которую пришлось пойти в моей бейзлайне (а на другое не хватило времени).

4.2.2 Обучение

В качестве базовых моделей я взял bert-base (80m) и DeBERTa-v3-base (110m).



(a) BERT-base (batch size 32)



(b) DeBERTa-v3-base (batch size 16)

Рис. 1: Кривые обучения verifier-моделей

Обе модели демонстрируют стабильное убывание val loss и рост val accuracy на протяжении всех 5 эпох, что говорит об отсутствии переобучения. BERT достиг val accuracy $\approx 97,9\%$, DeBERTa — $\approx 96,6\%$, при этом обучалась она почти в 3 раза дольше по числу шагов (около 160к против 60к). Это объясняется меньшим батчем (16 против 32), из-за чего при том же количестве примеров шагов получается вдвое больше, а сам DeBERTa ещё и архитектурно тяжелее за счёт disentangled attention — отсюда и суммарный проигрыш во времени.

Но смущать должно другое: оба результата 96% и 98% крайне высоки и являются результатом (по моему) лика в данных. Из-за отсутствия адекватных негативов, модели учатся не смотреть в evidence и валидировать claim, а по структуре факта claim гадать, является ли это чем-то абстрактным (nei) или нет (sup/ref), а там уже рыщет по evidence в поисках подсказок.

Я думаю, с этим можно побороться с помощью аугментаций (например, я сейчас пробую в качестве soft negative использовать в evidence результат чужого ретрива, а в качестве

hard negative - тертив топ20ого - топ25го чанков).

Но, к сожалению, у меня не хватает времени, чтобы довести свою идею до ума до дедлайна, поэтому имеем что имеем.

5 Итог

Результатом работы стал выкаченная в HF-spaces [демо](#) моей модельки, с которой можно поиграться.

Результатом я остаюсь недоволен, но планирую довести его до ума, так как сама концепция фактчекера мне понравилась.

Так же промежуточными результатами могут считаться просчитанные [первый](#) и [второй](#) датасеты с прогнанными эмбедингами для википедии, я много вынес для себя когда изучал информацию и готовил данные к нужному формату для выкатки на HF.